

نطاق المشروع الكامل

Project Management System (PMS)

وثيقة Scope — الميزات المستخدمة وآلية العمل العامة

React 19 · Laravel 10 · MySQL · Sanctum · Google Gemini
Multi-Tenant · Graduation Projects Platform

فهرس المحتويات

1. ملخص تنفيذي
2. هدف المشروع ونطاقه
3. أصحاب المصلحة والأدوار
4. البنية العامة وكيف يعمل النظام
5. الميزات ضمن النطاق (In Scope)
6. خارج النطاق والقيود (Out of Scope)
7. الذكاء الاصطناعي
8. الأمان والصلاحيات
9. التقنيات المستخدمة
10. نقاط للمناقشة العامة

1. ملخص تنفيذي

PMS (Project Management System / ByteHub) منصة ويب لإدارة مشاريع التخرج والمشاريع الجامعية ضمن بيئة متعددة الجامعات (Multi-Tenant). يربط الطلاب والمشرفين ومديري الجامعات في مساحة عمل واحدة لكل institution، مع عزل بيانات كامل بين الجامعات.

النظام يوفّر: إنشاء المشاريع، بناء الفريق عبر الدعوات، إدارة المهام، نقاش الفريق (تعليقات المشروع)، رفع الإصدارات/الملفات، تكامل GitHub، إشعارات فورية، لوحات تحكم حسب الدور، وميزتين للذكاء الاصطناعي (اقتراح أفكار مشاريع + توليد مهام).

باختصار للمناقشة: PMS ليس مجرد todo list — هو منصة collaboration كاملة لمشاريع التخرج، مع tenant isolation، workflow موافقة حسابات، وصلاحيات على مستوى المشروع.

2. هدف المشروع ونطاقه

2.1 الهدف

- تمكين الطلاب من إنشاء مشاريع التخرج وإدارة الفريق والمهام والملفات.
- تمكين المشرفين من متابعة المشاريع وقبول/رفض دعوات الإشراف.
- تمكين مدير الجامعة من إدارة المستخدمين والموافقة على الحسابات الجديدة.
- تمكين مدير المنصة (Super Admin) من إدارة الجامعات والمنصة ككل.
- دعم عدة جامعات على نفس المنصة دون تداخل بيانات.

2.2 نطاق المشروع (Scope Boundary)

خارج النطاق	ضمن النطاق
Real-time chat/WebSockets · تعليقات على مستوى المهمة في الواجهة · نظام تقييم/درجات · دفع إلكتروني · تطبيق موبايل native · microservices · Queue · background jobs لل Repository pattern · AI كامل	مصادقة وتسجيل · موافقة حسابات · إدارة مشاريع · دعوات · مهام · تعليقات مشروع · إصدارات ملفات · GitHub OAuth · إشعارات · لوحات تحكم · AI tasks · AI ideation · (عربي/إنجليزي) · Multi-tenant

3. أصحاب المصلحة والأدوار

الدور	الوصف	أهم الصلاحيات
Student (طالب)	صاحب مشروع أو عضو في فريق	إنشاء مشروع · دعوة مشرف/طلاب · مهام · تعليقات · إصدارات · AI ideation · AI tasks · GitHub
Supervisor (مشرف)	مشرف أكاديمي — قد ينتمي لعدة جامعات	قبول/رفض دعوات إشراف · متابعة مشاريع · مهام · تعليقات · مغادرة الإشراف
Admin (مدير جامعة)	مسؤول tenant واحد (جامعة واحدة)	إدارة مستخدمي الجامعة · موافقة/رفض حسابات · إنشاء مشاريع · طلبات كلمة المرور
Super Admin (مدير المنصة)	إدارة مركزية للمنصة	CRUD جامعات · جميع المستخدمين · جميع المشاريع · إحصائيات المنصة

ملاحظة: مدير الجامعة ليس جدولاً منفصلاً — هو مستخدم في `users` بدور `admin` و `university_id` محدد.

4. البنية العامة وكيف يعمل النظام

4.1 النمط المعماري

- Client-Server ثلاثي الطبقات:** MySQL → Laravel REST API (Business) → React SPA (Presentation) (Data).
- Monolith API + Service Layer** (ProjectService, TaskService, AITaskService, ...)
- Shared Database Multi-Tenancy:** جامعة واحدة = tenant؛ العزل عبر `university_id` + `TenantScope`.

4.2 تدفق عام للمستخدم

1. يفتح التطبيق → يسجل أو يدخل.
2. إن كان الحساب **pending** → شاشة انتظار (الملف الشخصي + خروج فقط).
3. إن كان **active** → لوحة تحكم حسب الدور.
4. يعمل على المشاريع عبر: قائمة مشاريع → تفاصيل مشروع (تبويبات).
5. كل عملية حساسة تمر عبر: Sanctum token + Role middleware + ProjectAccess.

4.3 صفحة تفاصيل المشروع (قلب النظام)

التبويب	الوظيفة
Overview	معلومات المشروع · التقدم · الأعضاء · GitHub commits
Invites	دعوة مشرف/طلاب (للمالك)
Tasks	CRUD مهام · نسبة إنجاز · توليد AI
Comments	محادثة فريق على مستوى المشروع
Releases & files	رفع/تعديل/حذف إصدارات · push GitHub
Activity log	سجل زمني لنشاطات المشروع (مهام · إصدارات · فريق)

5. الميزات ضمن النطاق (In Scope)

5.1 المصادقة والحسابات

الميزة	كيف تعمل بشكل عام
التسجيل	طالب أو مشرف يختار جامعة/جامعات · الحساب يبدأ pending · يُبلّغ مدير الجامعة
الدخول	Sanctum token يُخزّن في المتصفح · يُرسل مع كل طلب API
موافقة الحساب	مدير الجامعة يوافق/يرفض · المشرف: موافقة لكل جامعة على حدة
نسيت كلمة المرور	طلب مساعدة → المدير يصدر كلمة مؤقتة
GitHub OAuth	ربط/إلغاء ربط حساب GitHub لرفع الإصدارات

5.2 إدارة المشاريع

- إنشاء: طالب أو مدير جامعة · المشروع يرتبط تلقائياً بـ `university_id` .
- الوصول: المالك · المشرف · الأعضاء المقبولون · مدير الجامعة · Super Admin.
- التقدم: نسبة مئوية من المهام المكتملة.
- GitHub: مزامنة commits · عرض سجل commits.
- مغادرة الإشراف: المشرف يمكنه ترك المشروع.

5.3 الدعوات (Invitations)

- دعوة مشرف: المالك يختار من قائمة مشرفين متاحين → المشرف يقبل/يرفض.

- **دعوة طالب:** المالك يدعو طلاباً من نفس الجامعة → قبول/رفض → عضوية في `project_members`.

5.4 المهام (Tasks)

- حالات: `pending` · `in_progress` · `completed`.
- أي عضو بصلاحيّة وصول يمكنه إنشاء/تعديل/حذف.
- تغيير الحالة يُسجّل في Activity log ويُرسَل إشعار.
- نسبة الإنجاز تُحدّث تلقائياً بعد كل عملية.

5.5 التعليقات

- **على مستوى المشروع فقط** في واجهة المستخدم (team chat).
- إضافة · عرض · تعديل · حذف (للمالك أو صاحب التعليق).

5.6 الإصدارات (Project Versions / Releases)

- رفع ملف مع عنوان ووصف · تخزين في `storage`.
- قائمة الإصدارات · تعديل `metadata` · حذف.
- دفع الإصدار إلى GitHub (مالك المشروع + GitHub مربوط).

5.7 الإشعارات

- تُنشأ عند: مهمة جديدة · تغيير حالة · رفع إصدار · دعوة · موافقة حساب · ...
- عرض · `delete` · `mark read` · `unread badge`.

5.8 لوحات التحكم

الدور	محتوى Dashboard
Student	مشاريعي · دعوات · badges
Supervisor	مشاريع الإشراف · دعوات مشرف
Admin	إحصائيات الجامعة · طلبات معلقة · badges
Super Admin	إحصائيات المنصة · جامعات · مستخدمين · مشاريع

5.9 إدارة المستخدمين

- **مدير الجامعة:** CRUD طلاب/مشرفين · `password reset requests` · `pending approval`.
- **Super Admin:** CRUD جامعات · CRUD جميع المستخدمين · CRUD جميع المشاريع.

5.10 التوطين (i18n)

واجهة ثنائية اللغة (عربي RTL / English LTR) عبر `LanguageContext` وملفات `locales`.

6. خارج النطاق والقيود (Out of Scope)

- لا WebSockets — الإشعارات/polling عند فتح الصفحة.
- لا UI لتعليقات المهام (API موجود جزئياً في backend فقط).
- Queue = sync — طلبات AI تنفذ synchronously.
- لا Laravel Policies — الصلاحيات عبر ProjectAccess helper + Middleware.
- لا نظام تقييم/درجات/ratings في التطبيق الحالي.
- Version timeline API موجود لكن غير مستخدم في Frontend.

7. الذكاء الاصطناعي

الميزة	المستخدم	الآلية
Project Ideation	Student	إدخال اهتمامات → Gemini يقترح 3 أفكار مشاريع → حفظ bookmarks (حد أقصى)
AI Task Generation	Student (عضو مقبول)	وصف المشروع → Gemini يولد 10 مهام → تُحفظ rate limit · regenerate · <code>ai_generated=true</code>

التكامل: `AIIdeationService` · `AITaskService` · `GEMINI_API_KEY` في `.env`.

8. الأمان والصلاحيات

الطبقة	الآلية
Authentication	Laravel Sanctum — Bearer token
Authorization	RoleMiddleware (admin, student, ...) · user.status middleware
Tenant isolation	TenantScope على university_id · Models
Project access	ProjectAccess::canAccess — owner/supervisor/member/admin/super_admin
Rate limiting	AI endpoints throttled · password help throttled

9. التقنيات المستخدمة

طبقة	التقنية
Frontend	React 19 · Vite 7 · MUI 7 · React Router · Context API · i18n
Backend	Laravel 10 · PHP 8.1+ · Sanctum · Socialite (GitHub)
Database	MySQL · ~22 · business جدول
AI	Google Gemini API (HTTP)
Storage	Laravel public disk (versions/files)

10. نقاط للمناقشة العامة

أسئلة شائعة في المناقشة:

1. لماذا **Multi-Tenant shared DB**؟ — أبسط في النشر والصيانة لعدد جامعات محدود؛ العزل منطقي عبر `university_id`.
2. لماذا **Monolith** وليس **Microservices**؟ — حجم المشروع لا يبرر تعقيد `microservices`.
3. كيف يُعزل **tenant**؟ — `TenantScope + university_id` على كل استعلام `tenant-scoped`.
4. **Workflow** المشرف **multi-university**؟ — `pivot supervisor_universities` · موافقة لكل جامعة · `refreshAccountStatusFromMemberships`.
5. **ProjectAccess vs Policies**؟ — `helper` مركزي أبسط للمشروع الحالي؛ `Policies` أنسب للتوسع.
6. دور **AI**؟ — مساعد `ideation` وتسريع `task breakdown` · ليس `autopilot` للمشروع.
7. **Activity log vs Version list**؟ — `Activity log` = كل الأحداث · `Releases tab` = ملفات مرفوعة فقط.
8. **Scalability limits?** — `sync AI` · `no cache` · `monolith` · `MVP/academic scope` مقبول للـ.

مرجع الوثائق المرافقة

- `PMS-ERD.dbm1` — مخطط قاعدة البيانات
- `PMS-ARCHITECTURE.html` — مخططات المعمارية
- `PMS-ARCHITECTURE-CHAPTER.pdf` — فصل المعمارية
- `PMS-ACTIVITY-DIAGRAM.pdf` — مخططات النشاط

PMS Project Scope · [python docs/generate_pms_scope.py](#)
For thesis discussion · features as implemented in codebase